

Auth System — Access & Refresh Tokens

Token Types

Token	TTL	Secret	Storage
Access	15 min	JWT_SECRET	localStorage (userToken / adminToken)
Refresh	30 days	JWT_REFRESH_SECRET	kha_refresh httpOnly cookie

Payload: sub (userId), email , role , jti (unique ID per token)

Flow

Login / Register

Routes: POST /api/v1/auth/login · POST /api/v1/auth/register

1. Backend signs both tokens
2. Access token → JSON response body → client saves to localStorage
3. Refresh token → httpOnly cookie (path=/api/v1/auth , sameSite=strict , 30d)
4. Refresh jti saved to Redis with 30d TTL

Authenticated Request

Any protected route — client sends Authorization: Bearer <accessToken>

1. auth.middleware.ts:41 verifies JWT signature
2. Checks blacklist:<jti> in Redis
3. If blocklisted → 401

Token Refresh

Route: POST /api/v1/auth/refresh

1. Browser auto-sends kha_refresh cookie
2. Backend verifies refresh JWT + checks jti exists in Redis
3. Old jti deleted from Redis (one-time use — rotation)

4. New access + refresh tokens issued, new cookie set

Logout

Route: POST /api/v1/auth/logout (*requires access token*)

1. Refresh `jti` deleted from Redis
2. Access `jti` added to `blocklist:<jti>` in Redis (TTL = remaining token lifetime)
3. Cookie cleared

API Routes

Method	Route	Auth Required	Description
POST	/api/v1/auth/register	No	Register + issue tokens
POST	/api/v1/auth/login	No	Login + issue tokens
POST	/api/v1/auth/refresh	Cookie	Rotate token pair
POST	/api/v1/auth/logout	Bearer	Revoke tokens
GET	/api/v1/auth/me	Bearer	Get current user

Key Files

Purpose	File
Sign / verify tokens	khaatiaraarot-api/src/utils/jwt.ts
Token service logic	khaatiaraarot-api/src/services/auth.service.ts
Cookie + response	khaatiaraarot-api/src/controllers/auth.controller.ts
Request auth guard	khaatiaraarot-api/src/middleware/auth.middleware.ts
API routes	khaatiaraarot-api/src/routes/auth.routes.ts
Client — user	khaatiaraarot-web/app/login/page.tsx , components/navbar.tsx
Client — admin	khaatiaraarot-web/app/admin/login/page.tsx , lib/adminApi.ts

Security Notes

- Refresh token `httpOnly` + `sameSite=strict` + `path=/api/v1/auth` — not JS-accessible, not sent to other routes
- Token rotation: each refresh consumes old `jti` — blocks replay attacks
- Blocklist: logout invalidates access token before natural expiry via Redis
- **XSS risk:** Access token stored in `localStorage` — exposed to any JS running on the page
- **Gap:** No client-side auto-refresh logic found — access token silently expires after 15 min